Basic Computer Architecture with Hardwired Control Unit by using Logisim

Course Name: Computer Architecture and Organization

Course Code: AIE 172

Lecturer: Dr.Emad Badry

Date:9/5/2026

Project Members

Ayatullah Mohamed Abdelhamid Mahmoud

Jana Mahmoud Mohamed Elhadidy

Hoor Amir Essa Ali

Yomna Ayman Ahmed Mohamed

Youssef Emad Galal El-Haddad

Ismail Mohamed Ismail Abdelkader

Mohamed Metwally Ahmed Ali El-Mogy

Ibrahim Hazem El-Sayed

Ahmed Mohamed Tharwat Mostafa

Mohamed Alaa Mohamed Husseiny

Table of Contents

Page	Title
3	Introduction
3	System Overview
4	Data Unit Design
9	Bus Design
12	ALU Design
16	Memory Design
19	Instruction Set Architecture
22	Control Unit Design And Implementation
27	Register Transfer Level
29	Input/Output System
30	Assembly & Machine Code
30	Simulation Results
32	conclusion

1.Introduction

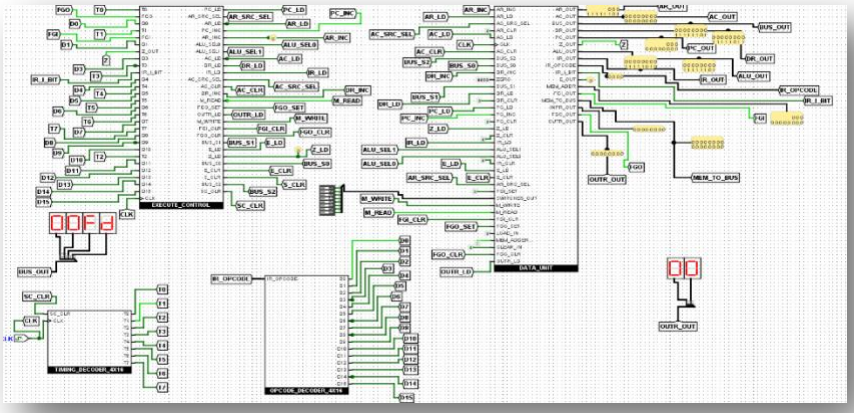
This project presents the design and implementation of a basic computer architecture using a hardwired control unit in Logisim Evolution. The system consists of a CPU, memory unit, and simple I/O interface. The CPU includes a data unit and a control unit that cooperate to execute instructions. The data unit contains registers, ALU, and buses for data processing, while the control unit generates control signals based on instructions and timing signals. The computer supports memory operations, arithmetic and logic instructions, branching, and I/O operations. The project demonstrates how computer components interact together to execute programs correctly.

2.System Overview

The system is composed of three main parts: CPU, memory, and I/O unit.
The CPU contains:

- Data Unit: registers, ALU, and common bus.
- Control Unit: instruction decoder, timing circuit, and control logic.

The memory size is 1024 × 16 bits and stores instructions and data.
The I/O interface uses switches as input devices and LEDs as output devices.
All components communicate through a 16-bit common bus controlled by the control unit.



Figure(1):Computer Organization Project Design

3.Data Unit Design

3.1 Overview

The data unit is the main part of the computer system responsible for data storage, transfer, and processing. It includes registers, memory, the ALU, and the internal bus, which work together under the control of the control unit to execute instructions.

3.2 Components of the Data Unit

The main components of the data unit are registers, flags, and other supporting units as shown in the following tables.

Registers

The system contains several registers, each one has a specific role in the execution process. These registers are used to store addresses, instructions, and data temporarily during execution.

Table 3.1: Registers of the Data Unit

Register	Size (bits)	Function
AR (Address Register)	11	Stores memory address
PC (Program Counter)	11	Holds address of next instruction
IR (Instruction Register)	16	Stores current instruction
DR (Data Register)	16	Holds data read from memory
AC (Accumulator)	16	Used for arithmetic and logic operations
INPR (Input Register)	8	Receives data from input device
OUTR (Output Register)	8	Sends data to output device

Flags

Flags are special bits that indicate certain conditions during the execution of instructions. These flags help the control unit in making decisions.

Table 3.2: Flags Used in the System

Flag	Description
Start/Stop (S)	If S=0 step execution of programs; Otherwise allow execution
Extension (E)	E=1 if this ADD operation produces a carry; E=0 Otherwise
Zero (Z)	Z=1 whenever the most recent ALU operation produces a zero result
Indirect (I)	I=1 whenever indirect addressing is used, and I=0 if direct addressing is used

Table 3.2: ALU and Memory Specifications

Component	Specification
ALU(Arithmetic Logic Unit)	Performs arithmetic and logic operations
Memory Unit	Bits 16 × 1024

3.3 Registers Design and Implementation

3.3.1 Introduction

This section focuses on designing and implementing the required registers in the basic computer using Logisim Evolution. Each register stores specific data such as instructions, addresses, or input/output values.

3.3.2 Design Idea

Registers are built using standard register components with control signals like Load, Increment, and Clear. Some registers also use multiplexers and adders.

3.3.3 Components Used

Registers (8-bit, 11-bit, 16-bit)

Multiplexers (MUX)

Adders

Constants (value = 1)

Clock

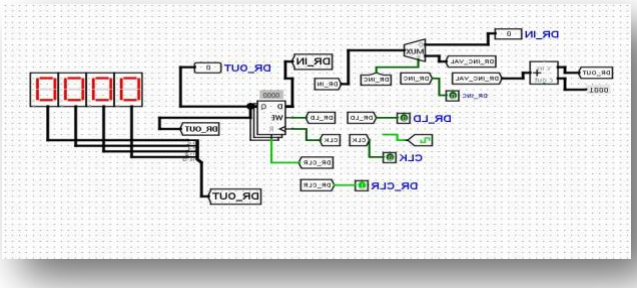
Input/Output Pins

splitter and displays

3.3.4 Registers Used

A) Data Register (DR)

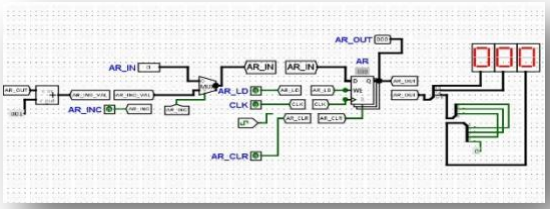
Stores data from memory or ALU.



Figure(3.1): Data Register (DR)

B) Address Register (AR)

Stores memory addresses.

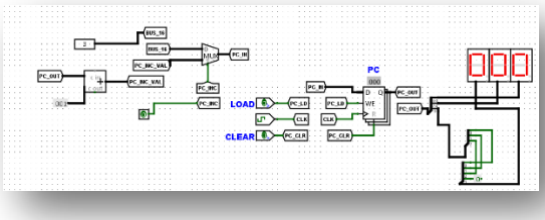


Figure(3.2): Address Register (AR)

C) Program Counter (PC)

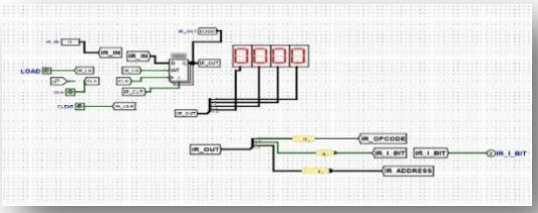
Holds the address of the next instruction

Figure(3.3): Program Counter (PC)



D) Instruction Register (IR)

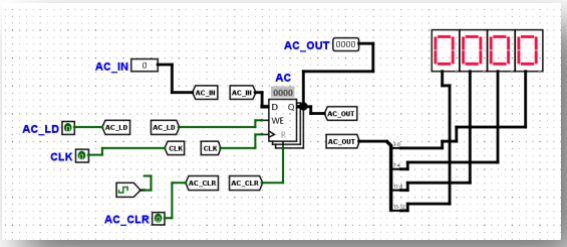
Stores the current instruction.



Figure(3.4): Instruction Register (IR)

E) Accumulator (AC)

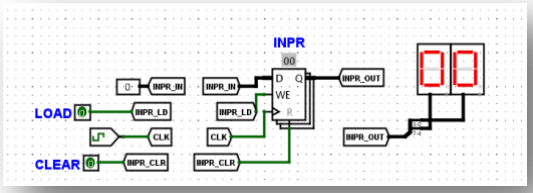
Stores ALU operation results.



Figure(3.5): Accumulator (AC)

F) Input Register (INPR)

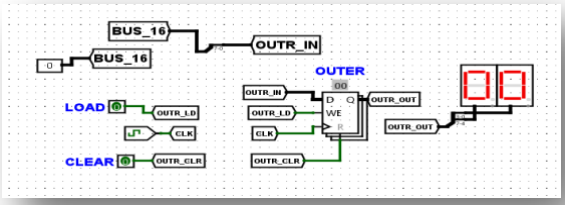
Receives input data.



Figure(3.6): Input Register(INNR)

G) Output Register (OUTR)

Sends output data to displays.

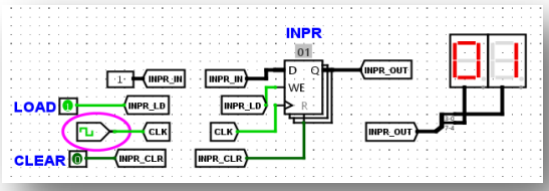


Figure(3.7): Output Register(OUTR)

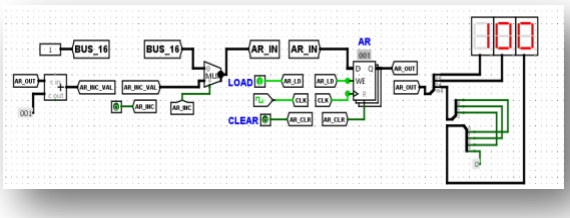
3.3.5 Testing and Simulation

All registers were tested using clock pulses and control signals. The Load operation stores data correctly, Increment increases the value by 1, and Clear resets the register to zero.

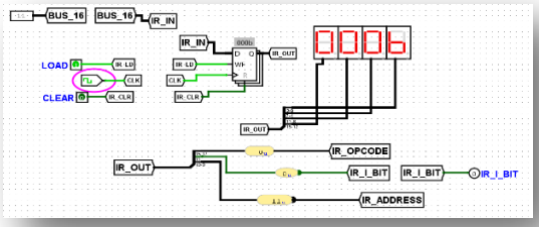
Register	Initial State	Action / Inputs	Expected Output (Hex)	Result
DR (16-bit)	0000	DR_IN = 1A2B, DR_LD = 1, Clock ↑	1A2B	Pass
DR (16-bit)	1A2B	DR_INC = 1, Clock ↑	1A2C	Pass
DR (16-bit)	1A2C	DR_CLR = 1, Clock ↑	0000	Pass
AR (11-bit)	000	AR_IN = 0FF, AR_LD = 1, Clock ↑	0FF	Pass
AR (11-bit)	0FF	AR_INC = 1, Clock ↑	100	Pass
AR (11-bit)	100	AR_CLR = 1, Clock ↑	000	Pass
PC (11-bit)	000	PC_IN = 020, PC_LD = 1, Clock ↑	020	Pass
PC (11-bit)	020	PC_INC = 1, Clock ↑	021	Pass
IR (16-bit)	0000	IR_IN = 7800, IR_LD = 1, Clock ↑	7800 (Opcode = 7, Addr = 800)	Pass
AC (16-bit)	0000	AC_IN = FFFF, AC_LD = 1, Clock ↑	FFFF	Pass
AC (16-bit)	FFFF	AC_CLR = 1, Clock ↑	0000	Pass
INPR (8-bit)	00	INPR_IN = A5, INPR_LD = 1, Clock ↑	A5	Pass
OUTR (8-bit)	00	OUTR_IN = 5A, OUTR_LD = 1, Clock ↑	5A	Pass



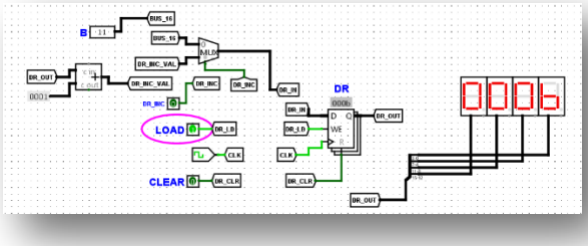
Figure(3.8): Test INPR



Figure(3.9): Test AR



Figure(3.10): Test IR



Figure(3.11): Test DR

3.3.6 Conclusion

All registers were successfully implemented and tested.

They are ready for integration with the full CPU system

4.Bus Design

4.1 Introduction

The 16-bit common data bus connects the main CPU components and allows data transfer between registers, memory, and other units under the control of the control unit.

4.2 Bus Architecture

The bus is built using:

- 8-to-1 Multiplexer (MUX)
- Bit Extenders
- Select Logic (3-bit control signals)

4.3 Multiplexer (MUX)

The multiplexer is the main component of the bus. It selects one of multiple input sources and forwards it to the bus output. The MUX used in this design has 8 inputs, each 16 bits wide, and one 16-bit output.

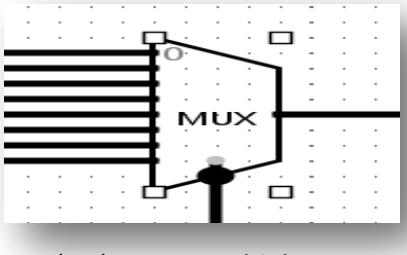


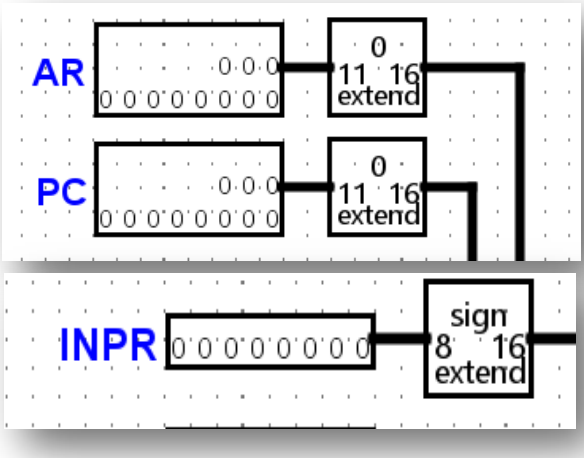
Figure (4.1): 8-to-1 Multiplexer Design

4.4 Bit Extenders

Some registers have smaller sizes than the 16-bit bus, so bit extenders are used:

- AR and PC: 11 bits
- INPR: 8 bits

These values are extended to 16 bits using zero extension.



Bit Extender Specifications			
Register	Input Bits	Output Bits	Extension
AR	11-bit	16-bit	Zero
PC	11-bit	16-bit	Zero
INPR	8-bit	16-bit	Zero

Figure(4.2): Bit Extension Circuit

4.5 Select Lines

Three select lines (BUS_S0, BUS_S1, BUS_S2) control which register places data on the bus. These signals are generated by the control unit.

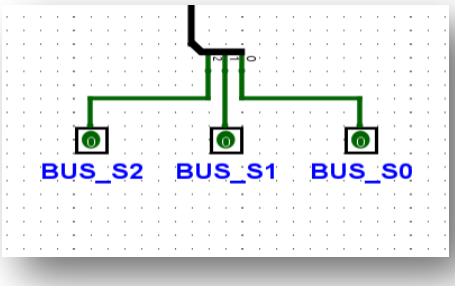


Figure (4.3): Select Lines Configuration

4.6 Bus Inputs

The bus receives data from different registers such as AR, PC, DR, AC, IR, and INPR through the multiplexer.

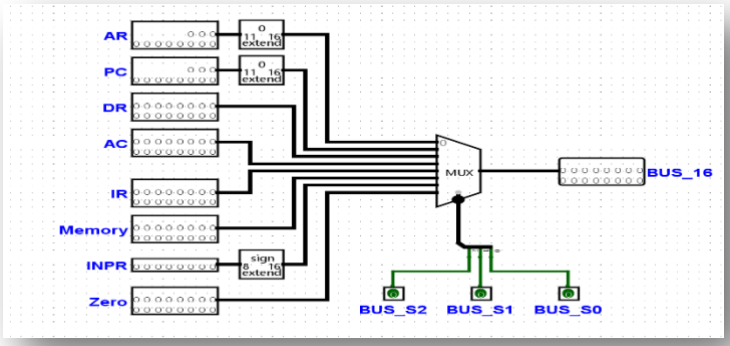


Figure (4.4): Complete Bus Connection

4.7 RTL Description

Bus operation is represented as:
BUS ← Selected Register

Bus Input Components (BUS_16)

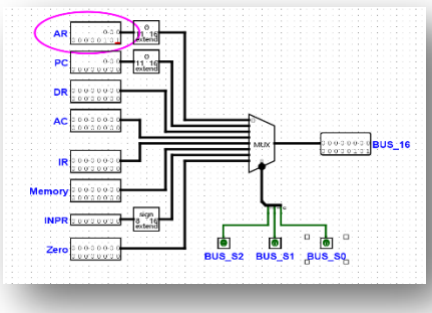
#	Component	Original Bits	On Bus	Select (S2S1S0)
0	AR	11-bit	16-bit (Zero Ext.)	000
1	PC	11-bit	16-bit (Zero Ext.)	001
2	DR	16-bit	16-bit	010
3	AC	16-bit	16-bit	011
4	IR	16-bit	16-bit	100
5	Memory	16-bit	16-bit	101
6	INPR	8-bit	16-bit (Zero Ext.)	110
7	Zero	16-bit	16-bit (all 0s)	111

Figure (4.5): RTL Description

4.8 Testing and Simulation

The bus was tested using different select values, and all tests showed correct data transfer between registers and the bus.

Figure (4.6): Test AR





4.9 Conclusion

The 16-bit common data bus was successfully designed and implemented using Logisim. The use of a multiplexer and control signals allowed efficient and flexible data transfer between system components.

5.ALU Design

5.1 Introduction

The ALU is a core component of the computer system responsible for performing arithmetic and logic operations. It works with registers and the bus to execute computations.

5.2 Main Role of the ALU

The ALU performs four main operations:

- ADD (Arithmetic addition)
- AND (Logical operation)
- CMA (Complement / NOT)
- PASS (Data transfer)

It also generates status flags:

- ZERO (Z)
- Carry (E)

5.3 ALU Operations and Selection Logic

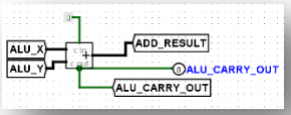
The ALU operation is controlled using two selection lines:
ALU_SEL1 and ALU_SEL0

ALU_SEL1	ALU_SEL0	Operation	Description
0	0	ADD	AC + DR
0	1	AND	AC ∧ DR
1	0	CMA/ NOT	NOT AC
1	1	PASS	Transfer input

5.4 Internal Components of the ALU

5.4.1 Adder

Performs 16-bit addition and produces a result and carry-out.



Figure(5.1): 16-bit Adder Circuit

5.4.2 AND Logic Block

Performs bitwise AND operation between inputs.

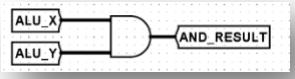


Figure (5.2): AND Logic Circuit

5.4.3 NOT Block

Inverts all bits of the input operand.

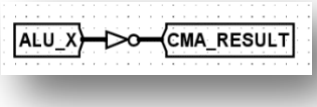


Figure (5.3): NOT Circuit

5.4.4 Multiplexer (MUX)

Selects one operation output to be the final ALU output based on control signals.

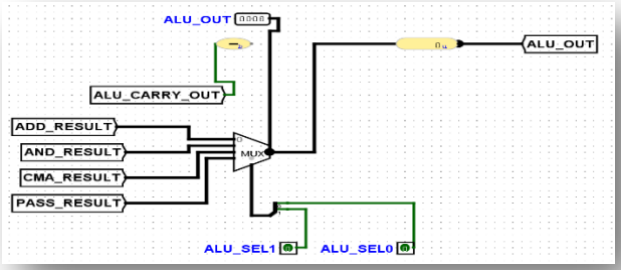


Figure (5.4): ALU Multiplexer

5.4.5 Flag Logic

- ZERO = 1 when result = 0

  ZERO = 0 otherwise

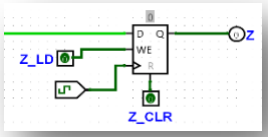


Figure (5.5): Flag Zero

- Carry = 1 when overflow occurs

  Carry = 0 otherwise

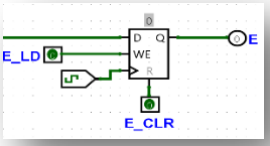


Figure (5.6): Flag Carry

5.5 Inputs and Outputs

5.5.1 Inputs

- AC (16-bit)
- DR (16-bit)
- ALU_SEL1, ALU_SEL0

5.5.2 Outputs

- ALU_OUT (16-bit)
- ZERO (Z)
- Carry (E)

5.6 Connections with the System

AC stores the result

DR provides the second operand

ALU output goes to the common bus

5.7 Flags Generated by the ALU

- **ZERO** is set when the result is 0
- **Carry** is set when overflow occurs

5.8 Testing and Simulation

The ALU was tested with ADD, AND, NOT, ZERO, and Carry cases, and all results were correct.

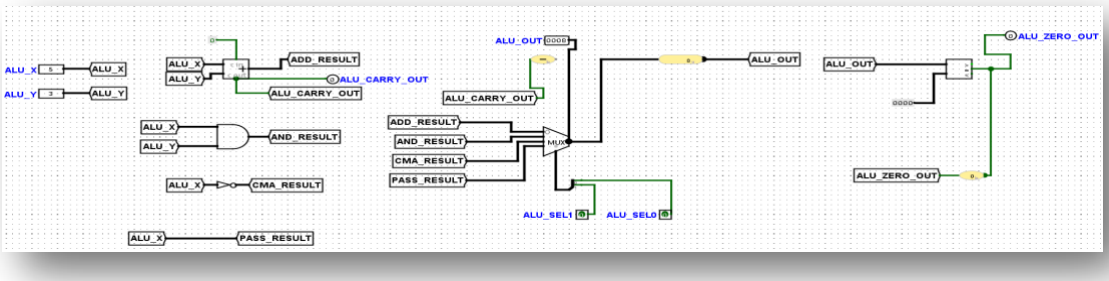


Figure (5.7): ADD Operation Test

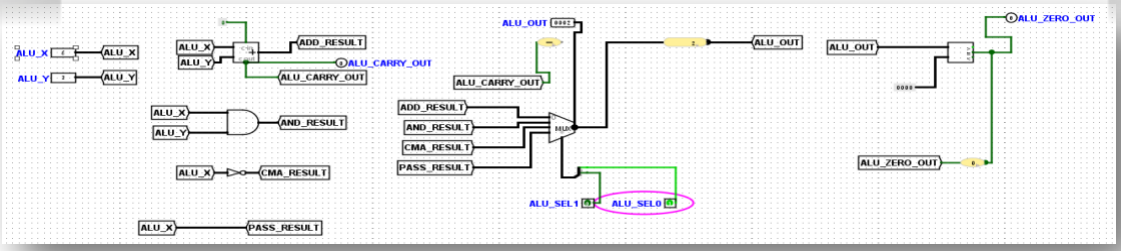


Figure (5.8): AND Operation Test

5.10 Conclusion

The ALU was successfully designed and tested. It performs all required operations correctly and generates accurate status flags.

6. Memory Design

6.1 Introduction

This module designs a **1024 × 16 memory unit** used to store both instructions and data. It supports instruction fetch, execution, and temporary data storage during program operation.

6.2 Memory Organization

The memory consists of **1024 locations**, each with **16 bits**.

- Address width: 10 bits (to access 1024 locations)
- Data width: 16 bits

6.3 Design Idea

The memory system is designed to support both instruction fetch and data operations using controlled signals and address selection logic.

Key features:

- Address is selected using a multiplexer between PC and AR
- Data flows through the common bus
- Controlled by read and write signals

6.4 Components Used

RAM (1024 × 16)
Multiplexer (Address Selection)
Common Bus (BUS_16)
Control Unit signals
Tunnels for signal organization

6.5 Memory Architecture

The memory system includes:

- Address input (MEM_ADDR)
- Data input (MEM_DIN)
- Data output (MEM_DOUT)
- Control signals (M_READ, M_WRITE)

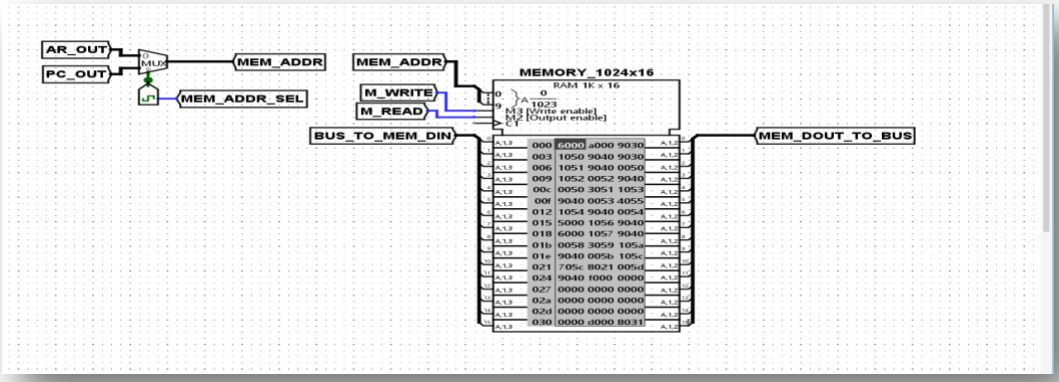


Figure (6.1): Memory Block Diagram

6.6 Address Selection (MUX)

The address is selected between **PC and AR** using a multiplexer controlled by MEM_ADDR_SEL.

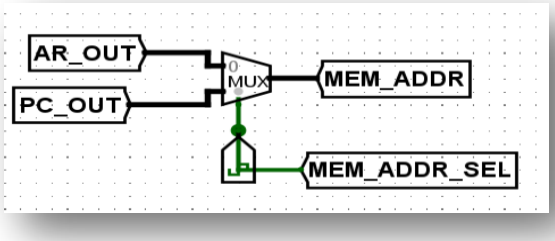


Figure (6.2): Address Selection using MUX

6.7 Read and Write Operations

- **Read:** Data is sent to the bus when M_READ = 1
- **Write:** Data from the bus is stored when M_WRITE = 1

6.8 Implementation

Implemented in Logisim using RAM and MUX.

The control unit manages timing for fetch (read) and store (write) operations.

6.9 Program Loading

- Instructions start at address 000
- Data starts at address 050

6.10 Testing and Simulation

The memory was tested using different scenarios:

- Read operations (LDA)
- Write operations (STA)
- Address switching between PC and AR

All tests confirmed correct functionality.

6.11 Signal Dictionary

Signal	Type	Function	Source/Destination
MEM_ADDR	Tunnel	Memory address input	Output of MUX (PC_OUT or AR_OUT)
MEM_DIN	Tunnel	Memory data input	Bus (BUS_TO_MEM_DIN)
MEM_DOUT	Tunnel	Memory data output	To bus **(**MEM_DOUT_TO_BUS)
M_READ	Control Signal	Activates memory read	Control Unit
M_WRITE	Control Signal	Activates memory write	Control Unit
BUS_TO_MEM_DIN	Tunnel	Bus → Memory data input	Bus → Memory

MEM_DOUT_TO_BUS	Tunnel	Memory → Bus data output	Memory → Bus
MUX (PC_OUT / AR_OUT)	Subcircuit	Selects address source	PC or AR
MEM_ADDR_SEL	Tunnel/Pin	Multiplexer selector	Control Unit

6.12 Conclusion

The memory module was successfully designed and tested. It supports correct storage of instructions and data with proper read and write control.

7. Instruction Set Architecture (ISA)

7.1 Introduction

ISA defines the set of instructions the CPU can execute and describes how instructions are structured and processed.

7.2 Instruction Format

Each instruction is **16 bits** and consists of:

- **Opcode (4 bits)** → specifies the operation
- **Address (11 bits)** → memory address
- **I bit (1 bit)** → addressing mode (direct or indirect)

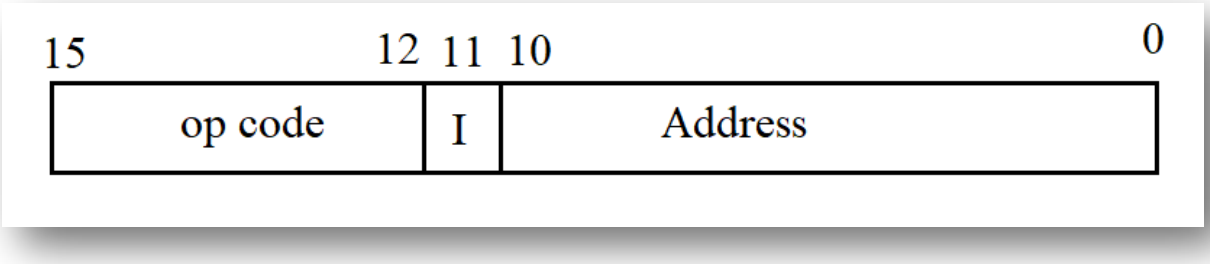


Figure (7.1): Instruction Format (16-bit structure)

7.3 Addressing Modes

Direct Addressing (I = 0) : The address field directly points to the memory location.

Indirect Addressing (I = 1): The address field points to a memory location that contains the effective address.

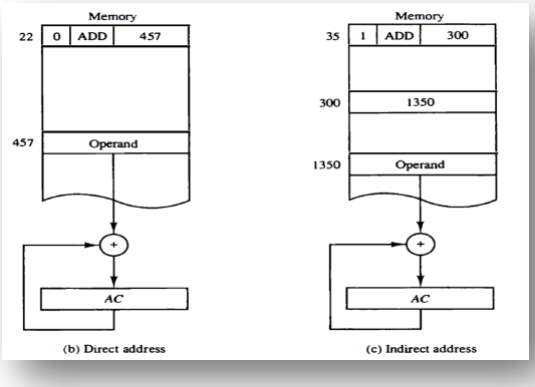


Figure (7.2): Direct vs Indirect Addressing

7.4 Instruction Set

Instruction	Opcode	Function
LDA	0000	AC ← M[EA]
STA	0001	M[EA] ← AC
ADD	0011	AC ← AC + M[EA]
AND	0100	AC ← AC ∧ M[EA]
CMA	0101	AC ← AC’
CLA	0110	AC ← 0
ISZ	0111	M[EA] ← M[EA] + 1
BUN	1000	PC ← EA
BSA	1001	M[EA] ← PC
CLE	1010	E ← 0
INP	1011	AC(0–7) ← INPR
OUT	1100	OUTR ← AC(0–7)
SKI	1101	Skip if FGI = 1
SKO	1110	Skip if FGO = 1
HLT	1111	Stop execution

7.5 Instruction Classification

Instructions are grouped into:

- **Memory Reference Instructions** (LDA, STA, ADD, AND, ISZ)
- **Register Reference Instructions** (CMA, CLA, CLE)
- **Branch Instructions** (BUN, BSA)
- **Input/Output Instructions** (INP, OUT, SKI, SKO)
- **Control Instructions** (HLT)

7.6 Execution Concept

Each instruction is executed in threesteps:

1. Fetch instruction from memory
2. Decode the opcode
3. Execute the operation

7.7 Example

LDA 050 loads the value from memory address 050 into the accumulator (AC).

7.8 Conclusion

The ISA defines how the processor interacts with memory and executes operations. It provides a structured way to control the system and ensures correct program execution.

8. Control Unit Design and Implementation

8.1 Instruction Decoder

8.1.1 Introduction

The Instruction Decoder is a key component of the Control Unit. It interprets the opcode stored in the Instruction Register (IR) and generates control signals for instruction execution.

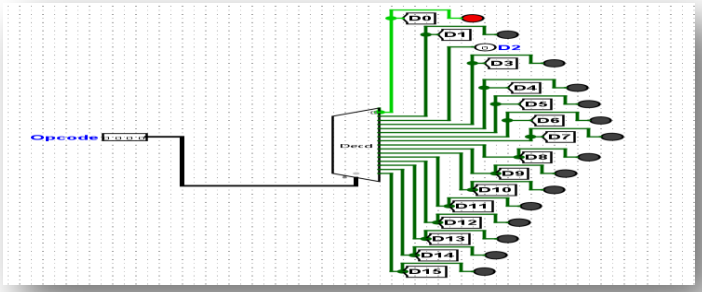


Figure (8.1): Instruction Decoder Block Diagram

8.1.2 Inputs to the Instruction Decoder

Opcode (4-bit) from IR

8.1.3 Outputs from the Instruction Decoder

Instruction Signals

INST_LDA
INST_STA
INST_ADD
INST_AND
INST_CMA
INST_CLA
INST_ISZ
INST_BUN
INST_BSA

8.1.4 Implementation

The instruction decoder is implemented using a **4×16 decoder** where the 4-bit opcode is used as input. Each output represents one instruction, and only one output is active at a time (**one-hot encoding**).

The decoder generates outputs (D0 ⟶ D15). Each output is mapped to a specific instruction.

D0 → INST_LDA
D1 → INST_STA
D2 → Not Used
D3 → INST_ADD
D4 → INST_AND
D5 → INST_CMA
D6 → INST_CLA
D7 → INST_ISZ
D8 → INST_BUN
D9 → INST_BSA
10 → INST_CLE
D11 → INST_INP
D12 → INST_OUT
D13 → INST_SKI
D14 → INST_SKO
D15 → INST_HLT

8.1.5 Instruction Mapping Table

Opcode	Signal	Instruction
0000	INST_LDA	Load AC
0001	INST_STA	Store AC
0011	INST_ADD	Add
0100	INST_AND	AND
0101	INST_CMA	Complement
0110	INST_CLA	Clear AC
0111	INST_ISZ	Increment
1000	INST_BUN	Branch
1001	INST_BSA	Branch Save
1010	INST_CLE	Clear E
1011	INST_INP	Input
1100	INST_OUT	Output
1101	INST_SKI	Skip Input
1110	INST_SKO	Skip Output
1111	INST_HLT	Halt

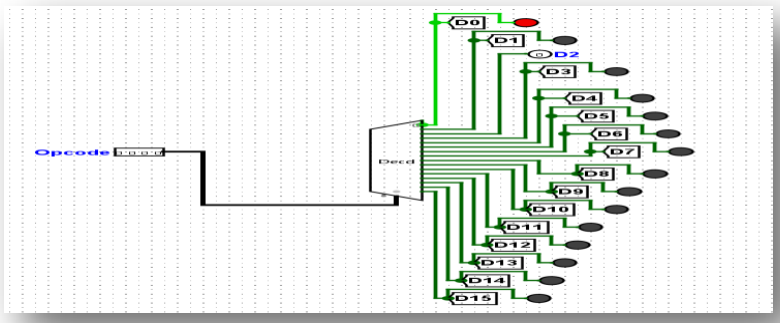


Figure (8.4): Instruction Mapping Table in Circuit

8.1.6 Testing and Verification

The decoder was tested by changing opcode values and observing outputs.

Test Cases:

- Opcode = 0000 → INST_LDA = 1
- Opcode = 0011 → INST_ADD = 1
- Opcode = 1111 → INST_HLT = 1

Verification Methods:

- LEDs monitoring
- Changing opcode manually
- Observing one-hot output behavior

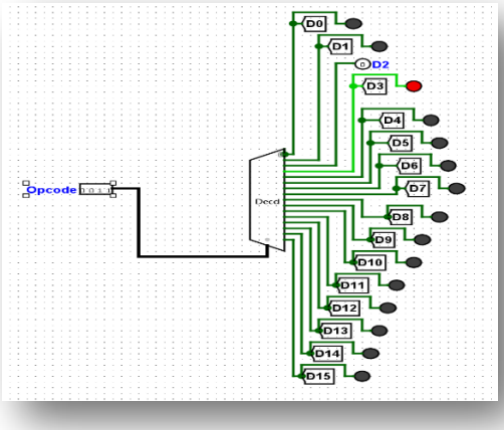


Figure (8.6): ADD Test

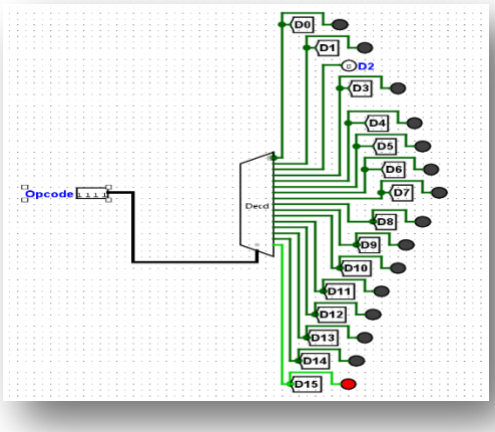


Figure (8.7): HLT Test

8.1.7 Challenges and Solutions

Challenges:

- Wiring complexity
- Signal naming confusion
- Many outputs to manage

Solutions:

- Using tunnels for organization
- Standard naming convention
- Testing each instruction separately

8.1.8 Conclusion

The Instruction Decoder was successfully implemented using a 4×16 decoder. It converts opcode values into one-hot instruction signals, simplifying control unit design and ensuring correct execution flow.

8.2 Timing Circuit

The timing circuit is part of the control unit that generates sequential signals **T0 to T15** to control instruction execution step by step.

It uses:

- Clock for synchronization
- 4-bit sequence counter (0–15)
- 4×16 decoder to generate timing signals
- Reset button / SC_CLR to return to T0
- LEDs for testing

Only one timing signal is active at a time, and signals activate sequentially from **T0 to T15**.

Conclusion:
The circuit works correctly and ensures proper sequencing and synchronization of CPU operations

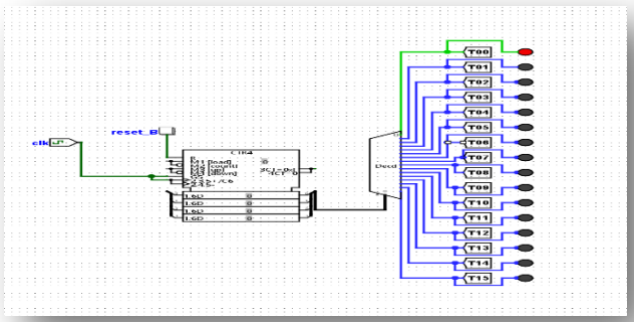


Figure (8.8): Timing Circuit Implementation in Logisim

8.3 Control Signal Generation

This stage generates all control signals needed to execute instructions based on:

- Instruction type (from decoder)
- Timing signals (T0–T7)
- Status flags (Z, FGI, FGO, I)

It produces signals for:

- Registers (AC, DR, PC, AR)
- Memory (read/write)
- ALU operations
- I/O operations
- Sequence control

Control signals are created using **Boolean logic (AND/OR gates)**, so each operation happens only at the correct time step.

Example (ADD at T5):
AC ← AC + DR using:

- ALU_ADD
- AC_LD
- E_LD

After finishing an instruction, **SC_CLR** resets the sequence counter to start the next instruction from T0.

Conclusion:
The control unit successfully coordinates all CPU operations by combining instruction decoding, timing, and control logic.

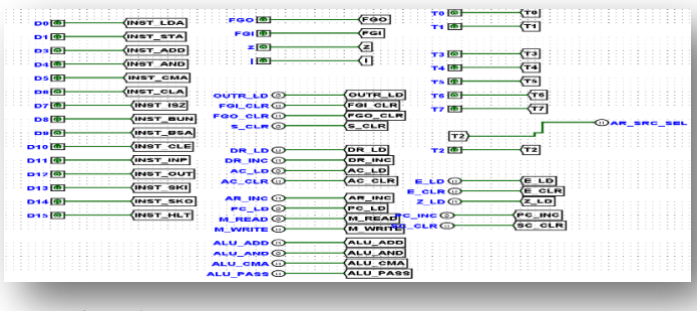


Figure (8.12): Control Unit and Control Signal Generation Design

9. Register Transfer Level (RTL)

9.1 Introduction

RTL describes how data moves between CPU registers and how instructions are executed using **micro-operations controlled by timing signals**. It shows the interaction between the **Data Unit and Control Unit**.

9.2 Data Unit RTL

The data unit (registers, ALU, memory) performs all data processing.

A) Fetch Cycle

T0: AR ← PC
T1: IR ← M[AR], PC ← PC + 1
T2: Decode IR

B) Example RTL Operations

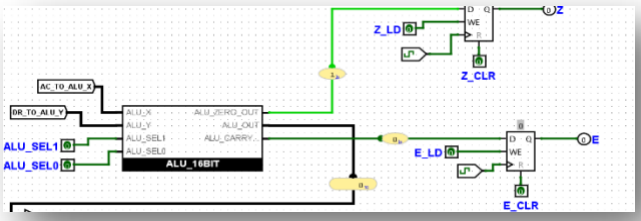
LDA Instruction

T3: AR ← IR(address)
T4: DR ← M[AR]
T5: AC ← DR

Step	Operation
T3	AR ← IR(address)
T4	DR ← M[AR]
T5	AC ← DR

ADD Instruction

T3: AR ← IR(address)
T4: DR ← M[AR]
T5: AC ← AC + DR, E ← Carry



Figure(9.1): ADD operation using ALU (AC + DR)

STA Instruction

T3: AR ← IR(address)
T4: M[AR] ← *AC*

BUN Instruction

T3: PC ← AR

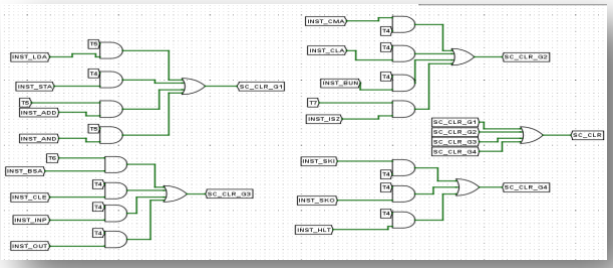
9.3 Control Unit RTL

The control unit generates signals to execute micro-operations using:

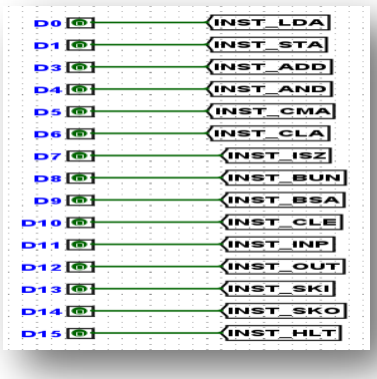
- Sequence Counter (SC) → generates timing signals T0, T1, T2…
- Instruction Decoder → identifies instruction type
- Control logic → activates required operations

Main control signals:

- LD (Load)
- INC (Increment)
- CLR (Clear)
- READ / WRITE (Memory control)



Figure(9.2): sequence counter



Figure(9.3): instruction decode

9.4 Execution Concept

Each instruction runs step-by-step based on:

- Opcode
- Timing signals
- Flags (Z, E, FGI, FGO)
- Indirect bit (I)

9.5 Conclusion

RTL clearly shows how data moves inside the CPU and how the control unit manages execution. The design was successfully implemented and verified using **Logisim**, confirming correct data transfer and control behavior.

10. Input/Output System

10.1 Introduction

The I/O unit enables communication between the CPU and external devices using **input switches and output displays**, controlled by status flags.

10.2 Input Unit

- Receives data through **8-bit switches**
- Stores data in **Input Register (INPR)**
- Data is loaded on clock edge
- **FGI flag = 1** when input data is ready

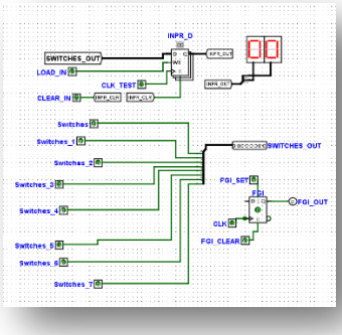


Figure (10.1): Input unit (Switches) → INPR with FGI flag

10.3 Output Unit

Sends data from CPU to display through **Output Register (OUTR)**
Output shown using LEDs or 7-segment display

FGO flag = 1 when output device is ready

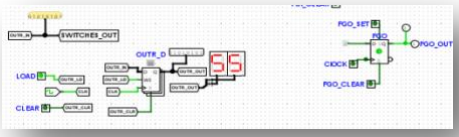


Figure (10.2): Output unit

10.4 Input and Output Flags

- FGI (Input Flag): set to 1 when input data is ready
- FGO (Output Flag): set to 1 when the output device is ready

These flags help the CPU decide when to read or write data.

Flag	Function
FGI	Input ready
FGO	Output ready

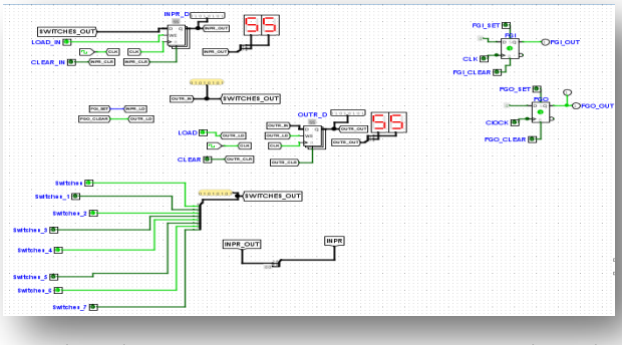
10.5 Operation Flow

Input: Switches → INPR → FGI = 1
Processing: CPU reads data when FGI = 1
Output: AC/Bus → OUTR → Display, FGO updated

10.6 Testing

The system was tested successfully:

- Data is correctly loaded into INPR using clock and load signal
- FGI is set when data is entered
- OUTR correctly displays output data
- FGO updates based on output status



Figure(10.3): Loading Data Into Input Register(INPR)

10.7 Conclusion

The I/O interface was successfully implemented and tested. It allows proper communication between the CPU and external devices using registers and control flags.

11. Assembly & Machine Code

11.1 Objective

The goal was to write an Assembly program, convert it to machine code, and load it into memory for execution on the basic computer system.

11.2 Instruction Set Used

The program used several instructions such as:

- **LDA, STA** → memory operations
- **ADD, AND, CMA, CLA** → arithmetic and logic operations
- **ISZ, BUN, BSA** → branching and looping
- **INP, OUT** → input/output operations
- **HLT** → stop execution

11.3 Assembly code

The Assembly program performs the following tasks:

- Initializes the system using **CLA** and **CLE**
- Reads two input values using **READ_INPUT**
- Displays input values using **DISPLAY_OUTPUT**
- Tests memory storage and retrieval
- Performs:
    - Addition (ADD)
    - Logical AND
    - Complement (CMA)
    - Clear accumulator (CLA)
    - Overflow test
- Executes a loop using **ISZ** and **BUN**
- Displays final result and stops using **HLT**

11.4 Subroutines

READ_INPUT

- Waits for input readiness using **SKI**
- Reads data using **INP**
- Returns to main program

DISPLAY_OUTPUT

- Waits for output readiness using **SKO**
- Sends data using **OUT**
- Returns to main program

Machine Code

Each Assembly instruction was converted into its corresponding **16-bit machine code** and stored in memory addresses for execution.

Conclusion

The program successfully demonstrated:

- Instruction execution
- Memory operations
- Arithmetic and logic processing
- Branching and looping
- Input/output communication

13. Conclusion

The project implemented a basic computer architecture in Logisim Evolution using a hardwired control unit. The system included registers, memory, ALU, control unit, and I/O interface. It successfully executed instructions, generated control signals, performed arithmetic and logic operations, and handled memory and I/O operations. The project improved understanding of CPU organization, RTL design, timing control, and datapath implementation.